

「货币战争·零和博弈」项目分析与优化方向

经过对《玩法复刻企划.pdf》与当前前端代码库的深度对比，当前项目已经具备了良好的前端表现层（UI）和基础的商店逻辑，但若要支撑企划书中 A8 难度下 12887% 的数值膨胀、每秒数千次的 DoT 结算以及复杂的卡池博弈，代码架构需要进行深度的重构与扩展。

以下是具体的优化方向与落地路径：

一、核心架构改造：引入战斗引擎（最重要的重构）

目前代码中 `startCombat` 只是一个 `setTimeout` 配合 `Math.random() > 0.3` 的占位符。企划书第三章明确要求使用 **ECS（实体-组件-系统）架构**，这是目前 React 状态驱动无法高效处理的。

- **痛点：**如果用 React 的 `setState` 或 Zustand 存储战斗中的每一次扣血、DoT 触发，会导致不可控制的 React 重渲染卡顿（Render Hell）。
- **优化方案：**
 1. **逻辑与表现分离（前后端分离模式）：**在前端代码中引入一个纯 TypeScript 编写的**独立战斗引擎类 (Combat Engine)**。
 2. **实现 AV (行动值) 驱动：**使用最小堆（Min-Heap）或时间轴（Timeline）数组来管理 10 名角色和敌人的行动顺序。每个刻度减扣 AV，归零者行动。
 3. **状态同步：**战斗引擎在内部以 60FPS 或极速推演，只在关键事件（如大招释放、血量低于阈值、战斗结束）时，通过回调函数或节流（Throttle）的方式将状态同步给 Zustand，从而更新 UI。

二、空间拓扑修正：10人场域重构

企划书（3.1 节）明确指出：场上是 **4个前台槽位 + 6个后台槽位**。而当前 `BattleBoard.tsx` 和 `gameStore.ts` 中写的是 8 个槽位（2排4列）。

- **优化方案：**
 1. 修改 `BoardState` 接口，将其拆分为 `onFieldSlots: (HeroInstance | null)[]`（长度 4）和 `offFieldSlots: (HeroInstance | null)[]`（长度 6）。
 2. 修改 `BattleBoard.tsx` 的 UI 布局。前排呈现为 4 个主要战斗位（模型大、有血条），后台呈现为 6 个支援位（模型小、主要提供光环或追加攻击）。
 3. 在拖拽/移动逻辑（`moveHero`）中，增加对前后台类型及等级人口上限的校验。

三、经济系统与卡池算法补全

企划书（第二章）的精髓在于“共享卡池压缩”和“动态概率矩阵”。当前代码 `generateShop` 是完全随机的，且没有卡池总量的概念。

- **优化方案：**
 1. **构建全局牌库 (Card Pool Model)：**初始化一个数组，包含特定数量的卡牌（例如：1费卡每种29张，5费卡每种10张）。

2. **卡池挤压逻辑**：当玩家购买英雄，从全局牌库中 pop 出该卡；当玩家出售或玩家阵亡，卡牌 push 回全局牌库。
3. **概率矩阵硬编码**：实现 2.1 节中的表格。在刷新商店时，先根据 `player.level` roll 出卡牌稀有度（例如 10 级有 25% 出 5 费），然后再从剩余的全局牌库中随机抽取对应稀有度的卡牌。
4. **利息算法修正**：当前的 `endCombat` 利息计算不够精准。需严格实现“每 10 金币 1 利息，基础上限 3”的逻辑，并预留可通过“投资策略”突破上限的参数接口。

四、羁绊（流派）的战斗级接入

当前 `bondCalculator.ts` 已经做得很棒了，能正确算出羁绊层数。但这只是 UI 层的展示，没有战斗中生效。

- **优化方案（结合企划书 4.1）：**

1. **DoT 体系**：在战斗引擎中添加 Buff/Debuff 管理器，处理每回合的真实伤害结算。
2. **仙舟（神君）**：当仙舟羁绊激活时，在战斗引擎的时间轴（Timeline）中显式插入一个名为“神君”的隐形 Entity，赋予其初始 AV，并根据仙舟羁绊层数修改其攻击段数。
3. **欢愉（阿哈时刻）**：引入“笑料点数（Punchline）”全局变量，后台角色行动时增加点数，满额触发全局 AOE。

五、补全 Roguelike 增量系统：装备与投资策略

当前代码完全缺少这两个维度的结构。

- **优化方案：**

1. **装备系统 (Gear System)**：在 `HeroInstance` 接口中增加 `equipment: string[]`。增加一个装备合成树谱谱（基础装备 A + B = 高级装备 C），并在 UI 层面增加装备拖拽给英雄的逻辑。
2. **节点推进系统 (Node Topology)**：当前只是 `node + 1`。需要创建一个地图组件，渲染树状或线性的节点路径（战斗、首领、补给、狂热等）。
3. **投资策略池 (Investment Strategies)**：实现企划书第六章的“三选一”事件。将 `investments.ts` 中的 `effect` 补充为真实的 Zustand 状态修改器（例如“降本增效”调用时，将 `maxInterest` 提升至 9，并给予 10 金币）。

六、专家顾问机制（独立卡池拓展）

- **优化方案：**

1. 根据企划书第七章，增加 `AdvisorState`，默认锁定。
2. 当玩家获得特定“专家邀请函”时，将银狼 LV.999、青雀等卡牌的实体推入**当前的全局牌库**中，使得玩家可以通过刷新商店刷出他们。

给开发者的短期 Action Items (接下来的两周目标)

如果这是你的个人项目或团队协作，建议按以下顺序开发以获得正反馈：

1. **修缮棋盘 (Day 1):** 将 BattleBoard 改为 4 (前台) + 6 (后台) 布局。
2. **卡池算法 (Day 2-3):** 实现带数量限制的公共卡池和基于玩家等级的随机刷新算法。
3. **战斗微内核 (Day 4-7):** 剥离 Zustand 的战斗逻辑, 写一个纯 JS/TS 的 CombatEngine 类, 实现最基础的 HP、攻击力、180AV 倒计时。在控制台(Console)能跑通一次 4v4 战斗。
4. **对接UI (Day 8-10):** 将 CombatEngine 的结果通过事件映射到 React UI, 实现血条扣减、伤害数字飘字。
5. **羁绊实装 (Day 11-14):** 挑最简单的两个羁绊 (如“星穹列车”加属性、“存护”加护盾) 在战斗引擎中跑通。